

Lab Computer Networks**Lab 1c****Socket Programming in Java: Chat Application**

- **Group:** CN-L1
 - **Team Members:**
 1. Nguyễn Minh Thái – MSSV: 2413133
 2. Nguyễn Lâm Sơn – MSSV: 2413012
 3. Nguyễn Quang Bảo – MSSV: 2410276
 - **Source Code Repository:** <https://github.com/Nguyen-Minh-Thai/Computer-Networks-Lab>
-

I. Objectives

1. Practice with Socket programming in Java
2. Build a simple chat application using client-server model
3. Multithreaded application in Java

II. Content**1. Socket programming in Java**

Consider the following code:

```
try {  
    // Get hostname by textual representation of IP address  
    InetAddress addr = InetAddress.getByName("127.0.0.1");  
    // Get hostname by a byte array containing the IP address  
    byte[] ipAddr = new byte[]{127, 0, 0, 1};  
    addr = InetAddress.getByAddress(ipAddr);  
    // Get the host name  
    String hostname = addr.getHostName();  
    // Get canonical host name  
    String hostnameCanonical = addr.getCanonicalHostName();  
}  
catch (UnknownHostException e) {  
}
```

Exercise 1:

Create a program that connects to a web server and downloads the homepage of this website to local computer.

Ex.: You can test your program with: *www.google.com*

Báo cáo & Bài làm: Exercise 1 – Tải Trang chủ Web

1. Phân tích & Nguyên lý hoạt động (DownloadHomepage.java):

- **Khởi tạo kết nối TCP:** Tạo socket kết nối tới cổng 80 (cổng HTTP mặc định) của máy chủ `www.google.com`.
- **Gửi bản tin HTTP Request:** Gửi yêu cầu HTTP GET theo phiên bản HTTP/1.0:

```
GET / HTTP/1.0\r\n
Host: www.google.com\r\n
User-Agent: Mozilla/5.0\r\n\r\n
```

Việc dùng HTTP/1.0 đảm bảo máy chủ tự động ngắt kết nối sau khi gửi hết dữ liệu, giúp luồng đọc nhận biết điểm kết thúc (-1).

- **Tách Header và lưu Body:** Đọc toàn bộ luồng byte vào bộ đệm, quét tìm chuỗi phân cách chuẩn `\r\n\r\n`. In toàn bộ HTTP Header ra màn hình và trích xuất phần Body lưu vào file `homepage.html`.

2. Đoạn mã nguồn thực thi chính:

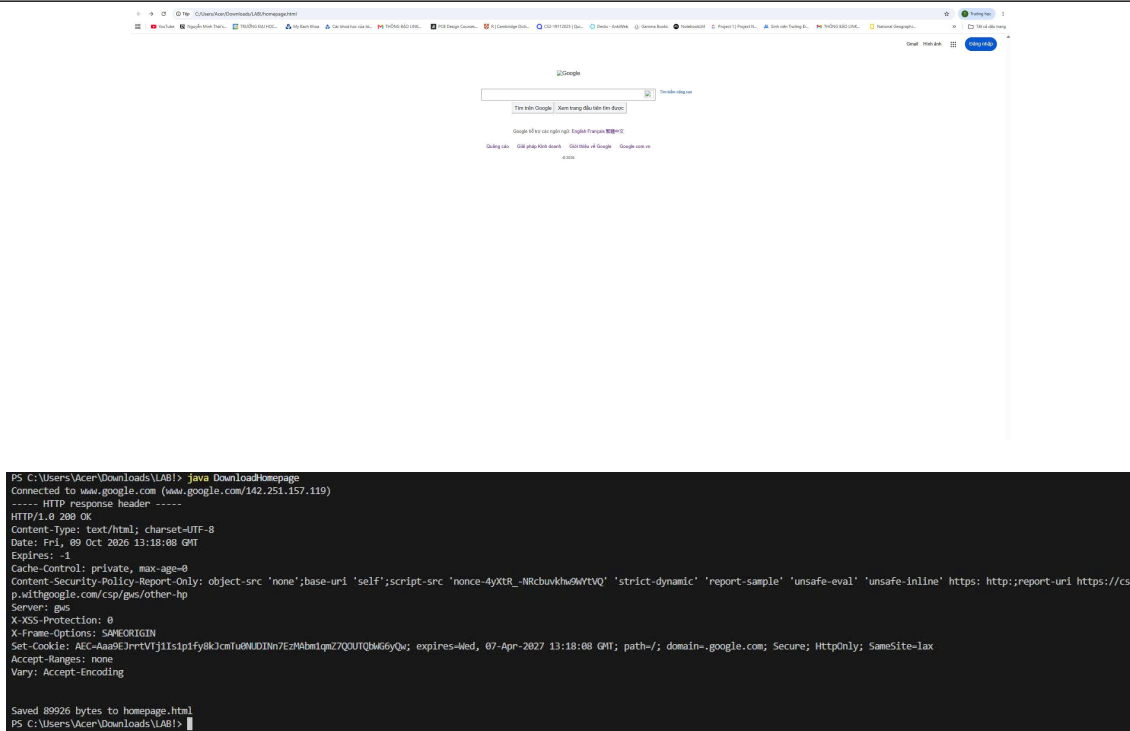
```
try (Socket socket = new Socket(host, 80)) {
    OutputStream out = socket.getOutputStream();
    String request = "GET / HTTP/1.0\r\nHost: " + host + "\r\nUser-Agent: "
        + "Mozilla/5.0\r\n\r\n";
    out.write(request.getBytes("ISO-8859-1"));
    out.flush();

    InputStream in = socket.getInputStream();
    ByteArrayOutputStream buf = new ByteArrayOutputStream();
    byte[] chunk = new byte[4096];
    int n;
    while ((n = in.read(chunk)) != -1) buf.write(chunk, 0, n);
    byte[] data = buf.toByteArray();

    // Tách header và body tại chuỗi \r\n\r\n
    int bodyStart = -1;
    for (int i = 0; i + 3 < data.length; i++) {
        if (data[i] == '\r' && data[i+1] == '\n' && data[i+2] == '\r' && data[i+3] == '\n') {
            bodyStart = i + 4;
            break;
        }
    }
    if (bodyStart >= 0) {
        try (FileOutputStream file = new FileOutputStream("homepage.html")) {
            file.write(data, bodyStart, data.length - bodyStart);
        }
    }
}
}
```

3. Kết quả thực nghiệm & Nhận xét:

- **Kết quả Terminal:** Kết nối thành công tới IP của Google (142.251.157.119:80), nhận mã phản hồi HTTP/1.0 200 OK, và lưu thành công 89,926 bytes vào file `homepage.html`.
- **Kết quả trình duyệt:** Khi mở file `homepage.html` offline, cấu trúc trang chủ Google hiển thị đầy đủ (ô tìm kiếm, nút bấm). Nút tìm kiếm không hoạt động và logo không tải được do các tài nguyên này dùng đường dẫn tương đối (`/search`), đây là đặc điểm tự nhiên khi tải mã nguồn HTML tĩnh bằng Socket.



Hình 1: (Trái) Log phản hồi HTTP trên Terminal; (Phải) File `homepage.html` hiển thị trên trình duyệt.

2. Develop a simple chat application using client-server model

Server application	Client application
Create a server socket	Identify server IP and port
Listen and wait for incoming connection request	Create a TCP socket to the server
Accept the connection request	Establish a connection to the server
Send/Receive data	Send/Receive data
Close connection	Close connection

Exercise 2: Design the user interface for the chat application

Báo cáo & Bài làm: Exercise 2 – Thiết kế Giao diện Ứng dụng Chat

1. Giao diện Server (ChatServer.java):

- Sử dụng Java Swing xây dựng cửa sổ điều khiển gồm: ô nhập Port lắng nghe (mặc định 9999), nút bấm chuyển đổi trạng thái (*Start / Stop*), và khung `JTextArea` có thanh cuộn hiển thị nhật ký kết nối và tin nhắn.

2. Giao diện Client (ChatClient.java):

- Form điều khiển chính (Chat Client – Main):** Ô nhập Host (localhost), Port (9999), Name (tên người dùng) và nút **"New connection"** để khởi tạo nhiều kết nối chat độc lập.
- Cửa sổ chat riêng (ChatWindow):** Mỗi kết nối mở ra một cửa sổ riêng biệt với khung hiển thị lịch sử chat và ô nhập tin nhắn bên dưới, tự động gửi tin khi nhấn phím **Enter**.

3. Multithread in Java

Create an application that has multiple threads running concurrently:

```
class PrimeRun implements Runnable {
    long minPrime;
    PrimeRun ( long minPrime ) {
        this.minPrime = minPrime;
    }
    public void run() {
        // compute primes larger than minPrime
        . . .
    }
}
PrimeRun p = new PrimeRun(143);
new Thread(p).start();
```

Báo cáo & Bài làm: Lập trình Đa luồng (PrimeRun.java)

- **Nguyên lý cài đặt:** Lớp PrimeRun cài đặt Runnable, tính toán các số nguyên tố lớn hơn minPrime và kiểm tra cờ ngắt !Thread.currentThread().isInterrupted() trong vòng lặp.
- **Kiểm chứng chạy song song:** Trong hàm main(), hai luồng T1 (mốc 143) và T2 (mốc 1000) được khởi chạy đồng thời. Kết quả trên terminal cho thấy các số nguyên tố của T1 và T2 được in xen kẽ nhau, chứng minh CPU đã phân bổ thời gian thực thi song song (concurrency). Sau 20ms, lệnh interrupt() được gọi và cả hai luồng dừng lại an toàn.

Stop a thread:

Try to use `Thread.interrupt()` as in the following code and see what happens.

```
public void stop() {
    running = false;
    this.interrupt();
}
public void run() {
    running = true;
    while (running) {
        Socket s = socket.accept();
        ...
    }
}
```

Phân tích & Giải thích: Cơ chế Dừng luồng khi bị tắc nghẽn ở accept()

1. Hiện tượng xảy ra khi gọi stop() dùng this.interrupt():

- **Kết luận:** Luồng Không thể dừng được Luồng tiếp tục bị kẹt và sống mãi mãi.

2. Bản chất nguyên nhân:

1. **Lỗi gọi bị chặn (Blocking I/O):** Phương thức `ServerSocket.accept()` là thao tác I/O đồng bộ bị chặn. Khi chưa có client nào kết nối, CPU treo luồng tại dòng này để đợi kết nối mạng.
2. **Cơ chế interrupt() bị vô hiệu:** Trong Java I/O truyền thống, `accept()` không ném ra ngoại lệ `InterruptedException`. Gọi `this.interrupt()` chỉ bật cờ trạng thái ngắt, nhưng hàm `accept()` hoàn toàn phớt lờ cờ này.
3. **Không bao giờ quay lại vòng lặp:** Do luồng đang bị kẹt cứng ở `accept()`, nó không thể thoát ra để quay lên đầu vòng lặp kiểm tra điều kiện `while (running)`. Vì vậy, gán `running = false` không có tác dụng.

3. Giải pháp chuẩn xác (StopTest.java):

- Để giải phóng luồng đang bị kẹt ở `accept()`, ta phải đóng socket từ luồng khác:

```
serverSocket.close();
```

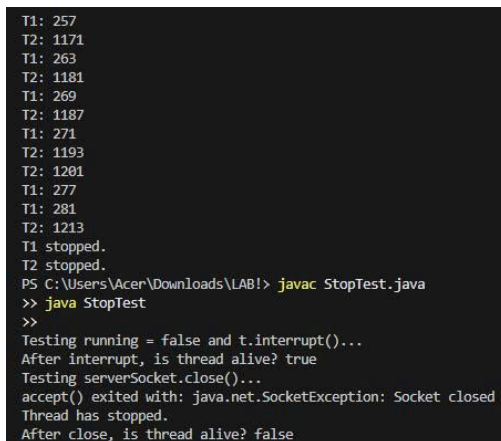
- Thao tác này ép buộc `accept()` giải phóng ngay lập tức và ném ra ngoại lệ:

```
java.net.SocketException: Socket closed
```

Khối catch sẽ bắt lấy ngoại lệ, luồng thoát khỏi vòng lặp và kết thúc an toàn.

4. Kết quả thực nghiệm in ra từ Terminal:

```
Testing running = false and t.interrupt()...
After interrupt, is thread alive? true
Testing serverSocket.close()...
accept() exited with: java.net.SocketException: Socket closed
Thread has stopped.
After close, is thread alive? false
```



```
T1: 257
T2: 1171
T1: 263
T2: 1181
T1: 269
T2: 1187
T1: 271
T2: 1193
T2: 1201
T1: 277
T1: 281
T2: 1213
T1 stopped.
T2 stopped.
PS C:\Users\Acer\Downloads\LAB1> javac StopTest.java
>> java StopTest
>>
Testing running = false and t.interrupt()...
After interrupt, is thread alive? true
Testing serverSocket.close()...
accept() exited with: java.net.SocketException: Socket closed
Thread has stopped.
After close, is thread alive? false
```

Hình 2: Kết quả kiểm chứng đa luồng PrimeRun và cơ chế dừng thread StopTest.

Multithread in Server Application:

- **Main window thread:** Chạy luồng giao diện Swing (Event Dispatch Thread).
- **Child thread (Listener):** Chạy vòng lặp `serverSocket.accept()` nhằm để đón kết nối mới mà không làm treo giao diện Server.
- **Grand-child thread (ClientHandler):** Tạo riêng cho từng Client kết nối để nhận dữ liệu và phát tin nhắn (broadcast).

Multithread in Client Application:

- **Main window thread:** Quản lý form nhập thông tin ban đầu.
- **Chat window thread:** Quản lý cửa sổ chat tương tác.
- **Grand-child thread (Reader):** Luồng chạy nền liên tục đọc `InputStream` từ server và cập nhật tin nhắn lên giao diện bằng `SwingUtilities.invokeLater()`.

Exercise 3:

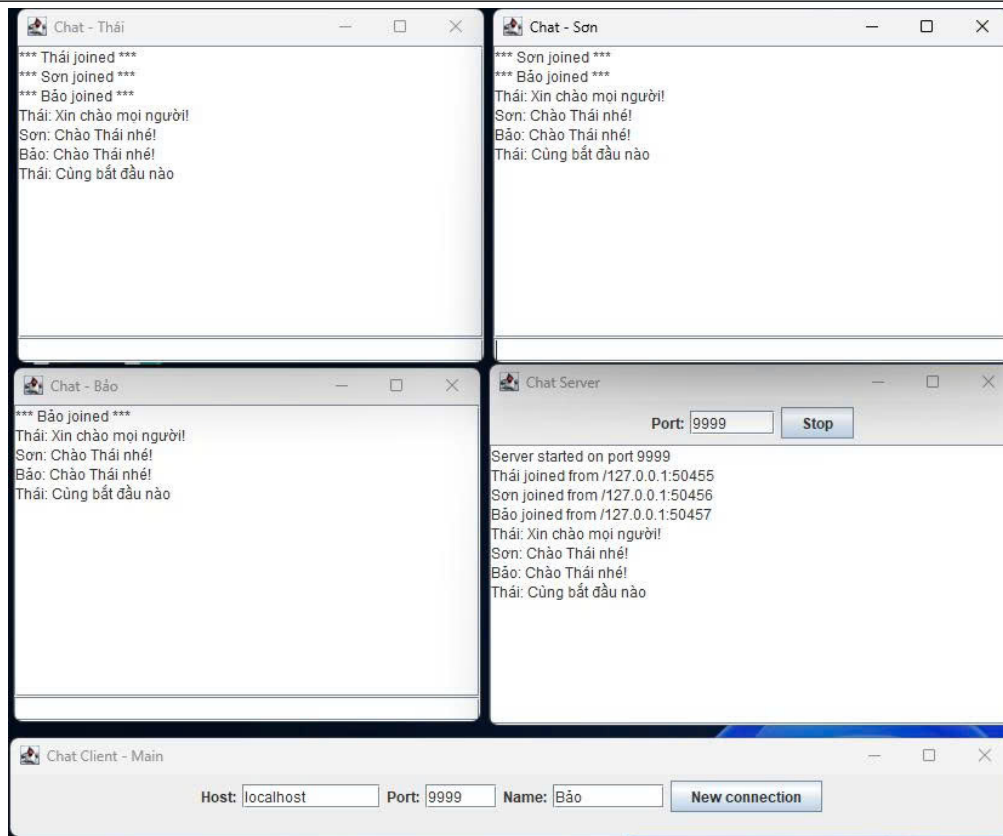
Using multithread programming model to make the chat application can talk to many different users concurrently.

Báo cáo & Bài làm: Exercise 3 – Ứng dụng Chat Đa luồng Đồng thời**1. Kiến trúc Đa luồng & Điểm nổi bật kỹ thuật:**

- **Cơ chế Broadcast an toàn:** Server quản lý danh sách client bằng `CopyOnWriteArrayList<ClientHandler>`. Khi một client gửi tin nhắn, Server chuyển tiếp tin nhắn thời gian thực đến toàn bộ các client đang online.
- **Đóng kết nối sạch sẽ:** Khi người dùng tắt cửa sổ chat, socket tự đóng an toàn, Server tự động xóa client khỏi danh sách và thông báo cho mọi người (***** [Name] left *****).

2. Kết quả Thử nghiệm Thực tế với 3 Thành viên Nhóm:

- **Kịch bản:** Khởi động ChatServer (Port 9999). Mở đồng thời 3 Client mang tên 3 thành viên: **Thái, Sơn, và Bảo**.
- **Kiểm chứng:** Khi các thành viên gửi tin nhắn (ví dụ: *"Xin chào mọi người!"*, *"Chào Thái nhé!"*, *"Cùng bắt đầu nào"*), tin nhắn lập tức xuất hiện đồng bộ ở cả 3 cửa sổ và trên bảng Log của Server theo thời gian thực mà không gặp bất kỳ lỗi xung đột hay trễ luồng nào.



Hình 3: Giao diện ứng dụng Chat đa luồng thời gian thực với Server và 3 client.